Day 9

1. Build class Wallet (private balance) and class WalletAuditor as a friend class.

Wallet exposes deposit/withdraw with validation.

WalletAuditor::report(const Wallet&) reads internals.

Show friendship is not transitive: a third class cannot access Wallet despite WalletAuditor friendship.

```
#include <iostream>

using namespace std;


class WalletAuditor;   // forward declaration


class Wallet {

private:

   double balance;


public:

   Wallet(double b = 0) {

      balance = b;

   }


   void deposit(double amount) {

      if (amount > 0) {

         balance += amount;

         cout << "Deposited: " << amount << endl;

      } else {

         cout << "Invalid deposit amount\n";

      }

   }
```

```
    void withdraw(double amount) {
        if (amount > 0 && amount <= balance) {
            balance -= amount;
            cout << "Withdrawn: " << amount << endl;
        } else {
            cout << "Invalid withdrawal amount\n";
        }
    }

    friend class WalletAuditor;
};

class WalletAuditor {
public:
    void report(const Wallet& w) {
        cout << "Auditor Report -> Balance = " << w.balance << endl;
    }
};

class ThirdClass {
public:
    void tryAccess(const Wallet& w) {
        // cout << w.balance;   // ERROR: cannot access private member
        cout << "ThirdClass cannot access Wallet private balance directly.\n";
    }
};
```

```
int main() {
    Wallet w(1000);

    w.deposit(500);
    w.withdraw(200);

    WalletAuditor auditor;
    auditor.report(w);

    ThirdClass t;
    t.tryAccess(w);

    return 0;
}
```

Output:

```
D:\cpp\1.exe

Deposited: 500
Withdrawn: 200
Auditor Report -> Balance = 1300
ThirdClass cannot access Wallet private balance directly.

--------------------------------
Process exited after 2.596 seconds with return value 0
Press any key to continue . . .
```

2. Create class Media with virtual play(). Derive Audio and Video overriding play().

Store heterogeneous objects in std::vector<std::unique_ptr<Media>> and invoke play() at runtime. Add virtual destructor and demonstrate.

```
#include <iostream>

#include <vector>

#include <memory>
```

```
using namespace std;

class Media {
public:
   virtual void play() {
      cout << "Playing media\n";
   }

   virtual ~Media() {
      cout << "Media destructor called\n";
   }
};

class Audio : public Media {
public:
   void play() override {
      cout << "Playing audio\n";
   }

   ~Audio() {
      cout << "Audio destructor called\n";
   }
};

class Video : public Media {
public:
   void play() override {
      cout << "Playing video\n";
```

```
    }

    ~Video() {
        cout << "Video destructor called\n";
    }
};

int main() {
    vector<unique_ptr<Media>> items;

    items.push_back(make_unique<Audio>());
    items.push_back(make_unique<Video>());
    items.push_back(make_unique<Audio>());

    cout << "Runtime polymorphism:\n";
    for (auto& item : items) {
        item->play();
    }

    cout << "Program ending, destructors will be called automatically:\n";
    return 0;
}
```

Output:

```
Runtime polymorphism:
Playing audio
Playing video
Playing audio
Program ending, destructors will be called automatically:
Audio destructor called
Media destructor called
Video destructor called
Media destructor called
Audio destructor called
Media destructor called

--------------------------------
Process exited after 1.812 seconds with return value 0
Press any key to continue . . .
```

3. Build class Rectangle with private width/height. Provide setters that enforce > O and throw on invalid input. Show attempted invariant violation from main() and how encapsulation prevents it.

```
#include <iostream>

#include <stdexcept>

using namespace std;


class Rectangle {

private:

   double width;

   double height;


public:

   Rectangle() {

      width = 1;

      height = 1;

   }


   void setWidth(double w) {
```

```
    if (w <= 0) {
        throw invalid_argument("Width must be greater than 0");
    }
    width = w;
  }

  void setHeight(double h) {
    if (h <= 0) {
        throw invalid_argument("Height must be greater than 0");
    }
    height = h;
  }

  void show() const {
    cout << "Width = " << width << ", Height = " << height << endl;
  }
};

int main() {
  Rectangle r;

  try {
    r.setWidth(10);
    r.setHeight(5);
    r.show();

    cout << "Trying invalid value...\n";
    r.setHeight(-3);   // invalid
```

```
    }
    catch (exception& e) {
        cout << "Exception caught: " << e.what() << endl;
    }

    // r.width = -5;   // ERROR: private member, cannot access directly

    return 0;
}
```

Output:

```
Width = 10, Height = 5
Trying invalid value...
Exception caught: Height must be greater than 0

--------------------------------
Process exited after 1.82 seconds with return value 0
Press any key to continue . . .
```

4.In a MathOps utility, overload pow(int,int) and pow(double, double) (compile time). In a Function hierarchy, override evaluate(double) (runtime). Write a driver that shows resolution differences at compile vs. run time.

```
#include <iostream>
#include <cmath>
using namespace std;

class MathOps {
public:
    static int pow(int a, int b) {
        int result = 1;
        for (int i = 0; i < b; i++) {
            result *= a;
        }
```

```
        return result;
    }

    static double pow(double a, double b) {
        return std::pow(a, b);
    }
};

class Function {
public:
    virtual double evaluate(double x) {
        return x;
    }

    virtual ~Function() {}
};

class SquareFunction : public Function {
public:
    double evaluate(double x) override {
        return x * x;
    }
};

class CubeFunction : public Function {
public:
    double evaluate(double x) override {
        return x * x * x;
```

```
    }
};

int main() {
    cout << "Compile-time polymorphism:\n";
    cout << "pow(2, 3) = " << MathOps::pow(2, 3) << endl;
    cout << "pow(2.5, 2.0) = " << MathOps::pow(2.5, 2.0) << endl;

    cout << "\nRuntime polymorphism:\n";
    Function* f1 = new SquareFunction();
    Function* f2 = new CubeFunction();

    cout << "SquareFunction evaluate(3) = " << f1->evaluate(3) << endl;
    cout << "CubeFunction evaluate(3) = " << f2->evaluate(3) << endl;

    delete f1;
    delete f2;

    return 0;
}
```

Output:

```
Compile-time polymorphism:
pow(2, 3) = 8
pow(2.5, 2.0) = 6.25

Runtime polymorphism:
SquareFunction evaluate(3) = 9
CubeFunction evaluate(3) = 27

--------------------------------
Process exited after 1.782 seconds with return value 0
Press any key to continue . . .
```

5.Create an array of pointers to strings and print them using pointer dereferencing.

```
#include <iostream>

#include <string>

using namespace std;


int main() {

   string s1 = "Apple";

   string s2 = "Banana";

   string s3 = "Orange";


   string* arr[3];

   arr[0] = &s1;

   arr[1] = &s2;

   arr[2] = &s3;


   cout << "Strings are:\n";

   for (int i = 0; i < 3; i++) {

      cout << *arr[i] << endl;

   }
```

```
    return 0;
}
```

Output:

```
Strings are:
Apple
Banana
Orange

--------------------------------
Process exited after 2.735 seconds with return value 0
Press any key to continue . . .
```

6. Demonstrate a program that uses a pointer to a pointer to modify a variable's value.

```
#include <iostream>
using namespace std;

int main() {
    int x = 10;
    int* p = &x;
    int** pp = &p;

    cout << "Before: " << x << endl;

    **pp = 50;

    cout << "After: " << x << endl;

    return 0;
}
```

Output:

```
Before: 10
After: 50

--------------------------------
Process exited after 1.853 seconds with return value 0
Press any key to continue . . .
```

7.Implement a menu-driven calculator where operations (+, -, *, /) are selected using function pointers.

```
#include <iostream>

using namespace std;


int add(int a, int b) {

   return a + b;

}


int sub(int a, int b) {

   return a - b;

}


int mul(int a, int b) {

   return a * b;

}


int divideNum(int a, int b) {

   if (b == 0) {

      cout << "Division by zero not allowed\n";

      return 0;

   }
```

```
    return a / b;
}

int main() {
    int choice, a, b;
    int (*funcPtr)(int, int);

    do {
        cout << "\nCalculator Menu\n";
        cout << "1. Addition\n";
        cout << "2. Subtraction\n";
        cout << "3. Multiplication\n";
        cout << "4. Division\n";
        cout << "5. Exit\n";
        cout << "Enter choice: ";
        cin >> choice;

        if (choice >= 1 && choice <= 4) {
            cout << "Enter two numbers: ";
            cin >> a >> b;
        }

        switch (choice) {
            case 1:
                funcPtr = add;
                cout << "Result = " << funcPtr(a, b) << endl;
                break;
            case 2:
```

```
            funcPtr = sub;
            cout << "Result = " << funcPtr(a, b) << endl;
            break;
        case 3:
            funcPtr = mul;
            cout << "Result = " << funcPtr(a, b) << endl;
            break;
        case 4:
            funcPtr = divideNum;
            cout << "Result = " << funcPtr(a, b) << endl;
            break;
        case 5:
            cout << "Exiting...\n";
            break;
        default:
            cout << "Invalid choice\n";
    }

  } while (choice != 5);

  return 0;
}
```

Output:

```
Calculator Menu
1. Addition
2. Subtraction
3. Multiplication
4. Division
5. Exit
Enter choice: 1
Enter two numbers: 2
3
Result = 5

Calculator Menu
1. Addition
2. Subtraction
3. Multiplication
4. Division
5. Exit
Enter choice: 2
Enter two numbers: 5
4
Result = 1

Calculator Menu
1. Addition
2. Subtraction
3. Multiplication
4. Division
5. Exit
Enter choice: 3
Enter two numbers: 5
4
Result = 20

Calculator Menu
1. Addition
2. Subtraction
3. Multiplication
4. Division
5. Exit
Enter choice: 4
```

```
Enter two numbers: 10
2
Result = 5

Calculator Menu
1. Addition
2. Subtraction
3. Multiplication
4. Division
5. Exit
Enter choice: 5
Exiting...

--------------------------------
Process exited after 25.53 seconds with return value 0
Press any key to continue . . .
```

8. Allocate an array of Student objects dynamically. Initialize and display their details.

```
#include <iostream>

using namespace std;


class Student {

public:

   int id;

   string name;

   float marks;


   void input() {
```

```
        cout << "Enter ID: ";
        cin >> id;
        cin.ignore();

        cout << "Enter Name: ";
        getline(cin, name);

        cout << "Enter Marks: ";
        cin >> marks;
    }

    void display() {
        cout << "ID: " << id
            << ", Name: " << name
            << ", Marks: " << marks << endl;
    }
};

int main() {
    int n;
    cout << "Enter number of students: ";
    cin >> n;

    Student* s = new Student[n];

    for (int i = 0; i < n; i++) {
        cout << "\nEnter details for student " << i + 1 << ":\n";
        s[i].input();
```

```
    }

    cout << "\nStudent Details:\n";
    for (int i = 0; i < n; i++) {
        s[i].display();
    }

    delete[] s;

    return 0;
}
```

Output:

```
Enter number of students: 3

Enter details for student 1:
Enter ID: 23
Enter Name: joyce
Enter Marks: 98

Enter details for student 2:
Enter ID: 45
Enter Name: sanjana
Enter Marks: 97

Enter details for student 3:
Enter ID: 27
Enter Name: nithya
Enter Marks: 99

Student Details:
ID: 23, Name: joyce, Marks: 98
ID: 45, Name: sanjana, Marks: 97
ID: 27, Name: nithya, Marks: 99

--------------------------------
Process exited after 29.2 seconds with return value 0
Press any key to continue . . .
```

9.Allocate a single object using new and delete it using delete. Demonstrate constructor and destructor calls.

```
#include <iostream>

using namespace std;


class Demo {

public:

   Demo() {

      cout << "Constructor called\n";

   }


   ~Demo() {

      cout << "Destructor called\n";

   }

};


int main() {

   Demo* d = new Demo();


   delete d;


   return 0;

}
```

Output:

D:\cpp\day 9\9.exe

```
Constructor called
Destructor called

--------------------------------
Process exited after 2.521 seconds with return value 0
Press any key to continue . . .
```